

Database Technologies - Self-Learning

MySQL Lab Exercises

1. Explain EXISTS vs IN performance comparison with an example

```
KD1_Rosh_98811> SELECT *  
  -> FROM emp e  
  -> WHERE EXISTS  
  -> (  
  ->     SELECT 1  
  ->     FROM dept d  
  ->     WHERE d.deptno=e.deptno  
  -> );
```

empno	ename	job	mgr	hire	sal	comm	deptno
7369	SMITH	CLERK	7902	1980-12-17	800.00	NULL	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600.00	300.00	30
7521	WARD	SALESMAN	7698	1981-02-22	1250.00	500.00	30
7566	JONES	MANAGER	7839	1981-04-02	2975.00	NULL	20
7654	MARTIN	SALESMAN	7698	1981-09-28	1250.00	1400.00	30
7698	BLAKE	MANAGER	7839	1981-05-01	2850.00	NULL	30
7788	SCOTT	ANALYST	7566	1982-12-09	3000.00	NULL	20
7844	TURNER	SALESMAN	7698	1981-09-08	1500.00	0.00	30
7876	ADAMS	CLERK	7788	1983-01-12	1100.00	NULL	20
7900	JAMES	CLERK	7698	1981-12-03	950.00	NULL	30
7902	FORD	ANALYST	7566	1981-12-03	3000.00	NULL	20
1000	NILESH	TRAINER	NULL	2004-01-01	2850.00	NULL	20

12 rows in set (0.976 sec)

```
KD1_Rosh_98811> SELECT *  
  -> FROM emp  
  -> WHERE deptno IN  
  -> (  
  ->     SELECT deptno  
  ->     FROM dept  
  -> );
```

empno	ename	job	mgr	hire	sal	comm	deptno
7369	SMITH	CLERK	7902	1980-12-17	800.00	NULL	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600.00	300.00	30
7521	WARD	SALESMAN	7698	1981-02-22	1250.00	500.00	30
7566	JONES	MANAGER	7839	1981-04-02	2975.00	NULL	20
7654	MARTIN	SALESMAN	7698	1981-09-28	1250.00	1400.00	30
7698	BLAKE	MANAGER	7839	1981-05-01	2850.00	NULL	30
7788	SCOTT	ANALYST	7566	1982-12-09	3000.00	NULL	20
7844	TURNER	SALESMAN	7698	1981-09-08	1500.00	0.00	30
7876	ADAMS	CLERK	7788	1983-01-12	1100.00	NULL	20
7900	JAMES	CLERK	7698	1981-12-03	950.00	NULL	30
7902	FORD	ANALYST	7566	1981-12-03	3000.00	NULL	20
1000	NILESH	TRAINER	NULL	2004-01-01	2850.00	NULL	20

12 rows in set (0.055 sec)

2. Implement a trigger to update the product price in cart, when price is updated in products table.

```

KD1_Rosh_98811> use classwork;
Database changed
KD1_Rosh_98811> select database();
+-----+
| database() |
+-----+
| classwork  |
+-----+
1 row in set (0.005 sec)

KD1_Rosh_98811> DROP TRIGGER IF EXISTS trig_update_cart_price;
Query OK, 0 rows affected (0.779 sec)

KD1_Rosh_98811> DELIMITER $$
KD1_Rosh_98811>
KD1_Rosh_98811> CREATE TRIGGER trg_product_price_update
-> AFTER UPDATE ON products
-> FOR EACH ROW
-> BEGIN
->     UPDATE cart
->     SET price=NEW.price
->     WHERE product_id=NEW.id;
-> END$$
Query OK, 0 rows affected (0.104 sec)

KD1_Rosh_98811>
KD1_Rosh_98811> DELIMITER ;
KD1_Rosh_98811> SELECT * FROM products;
Empty set (0.024 sec)

KD1_Rosh_98811> UPDATE products SET price=999 WHERE id=1;
Query OK, 0 rows affected (0.693 sec)
Rows matched: 0  Changed: 0  Warnings: 0

KD1_Rosh_98811> SELECT * FROM cart;
Empty set (0.005 sec)

KD1_Rosh_98811> DROP TABLE IF EXISTS emp;
Query OK, 0 rows affected (0.311 sec)

```

```

KD1_Rosh_98811> DROP TABLE IF EXISTS emp;
Query OK, 0 rows affected (0.311 sec)

KD1_Rosh_98811> DROP TABLE IF EXISTS dept;
Query OK, 0 rows affected (0.196 sec)

KD1_Rosh_98811> CREATE TABLE dept(
-> deptno INT PRIMARY KEY,
-> dname VARCHAR(20),
-> loc VARCHAR(20)
-> );
Query OK, 0 rows affected (1.003 sec)

KD1_Rosh_98811> CREATE TABLE emp(
-> empno INT PRIMARY KEY,
-> ename VARCHAR(20),
-> job VARCHAR(20),
-> sal DOUBLE,
-> deptno INT,
-> FOREIGN KEY(deptno) REFERENCES dept(deptno)
-> ON DELETE CASCADE
-> ON UPDATE CASCADE
-> );
Query OK, 0 rows affected (1.007 sec)

KD1_Rosh_98811> DELIMITER $$
KD1_Rosh_98811>
KD1_Rosh_98811> CREATE TRIGGER trg_product_price_update
-> AFTER UPDATE ON products
-> FOR EACH ROW
-> BEGIN
->     UPDATE cart
->     SET price = NEW.price
->     WHERE product_id = NEW.id;
-> END$$

```

3. Understand ON DELETE CASCADE, ON UPDATE CASCADE. Write emp, dept CREATE TABLE statements and apply CASCADE for all foreign keys

```
CREATE TABLE emp (  
empno INT PRIMARY KEY,  
ename VARCHAR(50) NOT NULL,  
job VARCHAR(50),  
sal DECIMAL(10,2),  
deptno INT,
```

```
CONSTRAINT fk_emp_dept  
FOREIGN KEY (deptno)  
REFERENCES dept(deptno)  
ON DELETE CASCADE  
ON UPDATE CASCADE  
);
```

```
INSERT INTO dept VALUES  
(10, 'ACCOUNTING', 'PUNE'),  
(20, 'RESEARCH', 'MUMBAI'),  
(30, 'SALES', 'NASHIK');
```

```
INSERT INTO emp VALUES  
(101, 'Amit', 'CLERK', 1200, 10),  
(102, 'Neha', 'MANAGER', 3500, 10),  
(103, 'Ravi', 'ANALYST', 2800, 20),  
(104, 'Pooja', 'CLERK', 1500, 30);
```

```
ON DELETE CASCADE
```

```
DELETE FROM dept  
WHERE deptno = 20
```

MongoDb Lab Exercises

1. Practice CRUD operations on Mongo.
1. Create two collections -- emp and dept.
2. Insert the emp and dept records.
3. Increase salary of dept 10 emps by 100.

4. Display emp name, job, deptno, sal in sorted order of deptno, job, sal (desc) for all emps whose sal more than 1000.
5. Delete the dept with id 40.

2. Learn grouping operations using Mongo aggregation pipeline

1. Display job wise total sal, max sal, min sal, and avg sal for all jobs other than PRESIDENT.
2. Display emp name, dept name, income (sal + comm) in sorted order of their income.

```
mongosh> use cdacdb
switched to db cdacdb
cdacdb> db.dept.insertMany([
...
  {
    deptno: 10,
    dname: "ACCOUNTING",
    loc: "PUNE"
  },
  {
    deptno: 20,
    dname: "RESEARCH",
    loc: "MUMBAI"
  },
  {
    deptno: 30,
    dname: "SALES",
    loc: "NASHIK"
  },
  {
    deptno: 40,
    dname: "OPERATIONS",
    loc: "DELHI"
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6aa7ee9dd14013632d6c4bd0'),
    '1': ObjectId('6aa7ee9dd14013632d6c4bd1'),
    '2': ObjectId('6aa7ee9dd14013632d6c4bd2'),
    '3': ObjectId('6aa7ee9dd14013632d6c4bd3')
  }
}
```

```
cdacdb> db.emp.insertMany([
...
  {
    empno: 7369,
    ename: "SMITH",
    job: "CLERK",
    sal: 800,
    deptno: 20
  },
  {
    empno: 7499,
    ename: "ALLEN",
    job: "SALESMAN",
    sal: 1600,
    deptno: 30
  },
  {
    empno: 7521,
    ename: "WARD",
    job: "SALESMAN",
    sal: 1250,
    deptno: 30
  },
  {
    empno: 7566,
    ename: "JONES",
    job: "MANAGER",
    sal: 2975,
    deptno: 20
  },
  {
    empno: 7839,
    ename: "KING",
    job: "PRESIDENT",
    sal: 5000,
    deptno: 10
  },
  {
    empno: 7900,
    ename: "JAMES",
    job: "CLERK",
    sal: 950
  }
])
```

```
cdacdb> db.emp.find()
[
  {
    _id: ObjectId('6aa7eea9d14013632d6c4bd4'),
    empno: 7369,
    ename: 'SMITH',
    job: 'CLERK',
    sal: 800,
    deptno: 20
  },
  {
    _id: ObjectId('6aa7eea9d14013632d6c4bd5'),
    empno: 7499,
    ename: 'ALLEN',
    job: 'SALESMAN',
    sal: 1600,
    deptno: 30
  },
  {
    _id: ObjectId('6aa7eea9d14013632d6c4bd6'),
    empno: 7521,
    ename: 'WARD',
    job: 'SALESMAN',
    sal: 1250,
    deptno: 30
  },
  {
    _id: ObjectId('6aa7eea9d14013632d6c4bd7'),
    empno: 7566,
    ename: 'JONES',
    job: 'MANAGER',
    sal: 2975,
    deptno: 20
  },
  {
    _id: ObjectId('6aa7eea9d14013632d6c4bd8'),
    empno: 7839,
    ename: 'KING',
    job: 'PRESIDENT',
    sal: 5000,
  }
]
```

```
cdacdb> db.emp.updateMany(
...   { deptno: 10 },
...   { $inc: { sal: 100 } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
```

```
cdacdb> db.emp.find(
...   { sal: { $gt: 1000 } },
...   {
...     _id: 0,
...     ename: 1,
...     job: 1,
...     deptno: 1,
...     sal: 1
...   }
... ).sort({
...   deptno: 1,
...   job: 1,
...   sal: -1
... })
[
  { ename: 'MILLER', job: 'CLERK', sal: 1400, deptno: 10 },
  { ename: 'CLARK', job: 'MANAGER', sal: 2550, deptno: 10 },
  { ename: 'KING', job: 'PRESIDENT', sal: 5100, deptno: 10 },
  { ename: 'JONES', job: 'MANAGER', sal: 2975, deptno: 20 },
  { ename: 'ALLEN', job: 'SALESMAN', sal: 1600, deptno: 30 },
  { ename: 'WARD', job: 'SALESMAN', sal: 1250, deptno: 30 }
]
```

```

cdacdb> db.dept.deleteOne({
...   deptno: 40
... })
{ acknowledged: true, deletedCount: 1 }
cdacdb> db.dept.find()
[
  {
    _id: ObjectId('6aa7ee9dd14013632d6c4bd0'),
    deptno: 10,
    dname: 'ACCOUNTING',
    loc: 'PUNE'
  },
  {
    _id: ObjectId('6aa7ee9dd14013632d6c4bd1'),
    deptno: 20,
    dname: 'RESEARCH',
    loc: 'MUMBAI'
  },
  {
    _id: ObjectId('6aa7ee9dd14013632d6c4bd2'),
    deptno: 30,
    dname: 'SALES',
    loc: 'NASHIK'
  }
]

```

```

cdacdb> db.emp.aggregate([
...   {
...     $match: {
...       job: { $ne: "PRESIDENT" }
...     },
...   },
...   {
...     $group: {
...       _id: "$job",
...       total_sal: {
...         $sum: "$sal"
...       },
...       max_sal: {
...         $max: "$sal"
...       },
...       min_sal: {
...         $min: "$sal"
...       },
...       avg_sal: {
...         $avg: "$sal"
...       }
...     }
...   },
...   {
...     $project: {
...       _id: 0,
...       job: "$_id",
...       total_sal: 1,
...       max_sal: 1,
...       min_sal: 1,
...       avg_sal: 1
...     }
...   }
... ])

```

```

cdacdb> db.emp.aggregate([
...   {
...     $lookup: {
...       from: "dept",
...       localField: "deptno",
...       foreignField: "deptno",
...       as: "department"
...     },
...   },
...   {
...     $unwind: "$department"
...   },
...   {
...     $project: {
...       _id: 0,
...       ename: 1,
...       dname: "$department.dname",
...       income: {
...         $add: [
...           "$sal",
...           { $ifNull: ["$comm", 0] }
...         ]
...       }
...     },
...   },
...   {
...     $sort: {
...       income: 1
...     }
...   }
... ])
[
  { ename: 'SMITH', dname: 'RESEARCH', income: 800 },
  { ename: 'JAMES', dname: 'SALES', income: 950 },
  { ename: 'WARD', dname: 'SALES', income: 1250 },
  { ename: 'MILLER', dname: 'ACCOUNTING', income: 1400 },

```

Leetcode Exercises

GROUP BY Problems

Q1

The screenshot shows the LeetCode interface for the problem "Customer Placing the Largest Number of Orders". The solution is written in MySQL and has been accepted. The runtime is 421 ms, beating 98.39% of other submissions. A bar chart shows the distribution of runtimes, with most submissions falling between 400ms and 600ms. The test result for Case 1 is shown as "Accepted" with a runtime of 65 ms.

Problem Description: Accepted x Editorial Solutions Submissions

Runtime: 421 ms | Beats 98.39%

Code:

```
1 # Write your MySQL query statement below
2 SELECT customer_number
3 FROM Orders
4 GROUP BY customer_number
5 ORDER BY COUNT(order_number) DESC
6 LIMIT 1;
```

Testcase: Accepted Runtime: 65 ms

Case 1:

Input:

order_number	customer_number
1	1
2	2
3	3
4	3

Q2

The screenshot shows the LeetCode interface for the problem "Biggest Single Number". The solution is written in MySQL and has been accepted. The runtime is 472 ms, beating 56.86% of other submissions. A bar chart shows the distribution of runtimes, with most submissions falling between 400ms and 600ms. The test result for Case 1 is shown as "Accepted" with a runtime of 111 ms.

Problem Description: Accepted x Editorial Solutions Submissions

Runtime: 472 ms | Beats 56.86%

Code:

```
1 # Write your MySQL query statement below
2 SELECT MAX(num) AS num
3 FROM MyNumbers
4 WHERE num IN
5 (
6     SELECT num
7     FROM MyNumbers
8     GROUP BY num
9     HAVING COUNT(num)=1
10 );
```

Testcase: Accepted Runtime: 111 ms

Case 1:

Input:

num
8
8
3
3
1
4

Q3

Problems - LeetCode | Game Play Analysis I - LeetCode

https://leetcode.com/problems/game-play-analysis-i/submissions/2143537214/

Problem List | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 12 / 12 testcases passed

ROSHANI DHEMBARE submitted at Sep 16, 2026 15:42

School's in. Leet's on. Get LeetCode Annual Premium for \$99!

Runtime: 486 ms | Beats 87.07%

Code | MySQL

```
1 # Write your MySQL query statement below
2 SELECT player_id, MIN(event_date) AS first_login
3 FROM Activity
4 GROUP BY player_id;
```

Testcase | Test Result

Accepted Runtime: 79 ms

Case 1

Input

Activity =

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-05-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

Q4

Problems - LeetCode | Students and Examinations - Leet

https://leetcode.com/problems/students-and-examinations/submissions/2143538235/

Problem List | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 15 / 15 testcases passed

ROSHANI DHEMBARE submitted at Sep 16, 2026 15:44

School's in. Leet's on. Get LeetCode Annual Premium for \$99!

Runtime: 1028 ms | Beats 77.68%

Code | MySQL

```
1 # Write your MySQL query statement below
2 SELECT s.student_id, s.student_name, sub.subject_name,
3 COUNT(e.subject_name) AS attended_exams
4 FROM Students s
5 CROSS JOIN Subjects sub
6 LEFT JOIN Examinations e
7 ON s.student_id = e.student_id
8 AND sub.subject_name = e.subject_name
9 GROUP BY s.student_id, s.student_name, sub.subject_name
10 ORDER BY s.student_id, sub.subject_name;
```

Testcase | Test Result

Accepted Runtime: 255 ms

Case 1

Input

Students =

student_id	student_name
1	Alice
2	Bob
13	John
6	Alex

Q5

The screenshot shows the LeetCode interface for the problem "Find Total Time Spent by Each Employee". The solution is accepted, with a runtime of 532 ms, beating 84.24% of other submissions. A bar chart shows the distribution of runtimes. The code is a MySQL query that calculates the total time spent by each employee for each event day.

Runtime: 532 ms | Beats 84.24%

Code (MySQL):

```
1 # Write your MySQL query statement below
2 SELECT event_day AS day, emp_id, SUM(out_time - in_time) AS total_time
3 FROM Employees
4 GROUP BY event_day, emp_id;
```

Testcase 1 (Accepted): Runtime: 75 ms

Input:

emp_id	event_day	in_time	out_time
1	2020-11-28	4	32
1	2020-11-28	55	200
1	2020-12-3	1	42
2	2020-11-28	3	33
2	2020-12-9	47	74

Q6

The screenshot shows the LeetCode interface for the problem "Actors and Directors Who Cooperated at Least Three Times". The solution is accepted, with a runtime of 433 ms, beating 33.55% of other submissions. A bar chart shows the distribution of runtimes. The code is a MySQL query that finds actors and directors who have cooperated at least three times.

Runtime: 433 ms | Beats 33.55%

Code (MySQL):

```
1 # Write your MySQL query statement below
2 SELECT actor_id, director_id
3 FROM ActorDirector
4 GROUP BY actor_id, director_id
5 HAVING COUNT(*) >= 3;
```

Testcase 1 (Accepted): Runtime: 89 ms

Input:

actor_id	director_id	timestamp
1	1	0
1	1	1
1	1	2
1	2	3
1	2	4
2	1	5

Q7

Problems - LeetCode

The Latest Login in 2020 - LeetCode

https://leetcode.com/problems/the-latest-login-in-2020/submissions/2143541563/

Problem List

Description Accepted Editorial Solutions Submissions

All Submissions

Accepted 24 / 24 testcases passed

ROSHANI DHEMBARE submitted at Sep 16, 2026 15:48

School's in. Leet's on. Get LeetCode Annual Premium for \$99!

Runtime

691 ms Beats 62.40%

Testcase Test Result

Accepted Runtime: 81 ms

Case 1

Input

Logins =

user_id	time_stamp
6	2020-06-30 15:06:07
6	2021-04-21 14:06:06
6	2019-03-07 00:18:15
8	2020-02-01 05:10:53
8	2020-12-30 00:46:50

Code | MySQL

```
1 # Write your MySQL query statement below
2 SELECT user_id, MAX(time_stamp) AS last_stamp
3 FROM Logins
4 WHERE time_stamp >= '2020-01-01'
5 AND time_stamp < '2021-01-01'
6 GROUP BY user_id;
```

Q8

Problems - LeetCode

Queries Quality and Percentage - LeetCode

https://leetcode.com/problems/queries-quality-and-percentage/submissions/2143543566/

Problem List

Description Accepted Editorial Solutions Submissions

All Submissions

Accepted 13 / 13 testcases passed

ROSHANI DHEMBARE submitted at Sep 16, 2026 15:50

School's in. Leet's on. Get LeetCode Annual Premium for \$99!

Runtime

396 ms Beats 58.75%

Testcase Test Result

Accepted Runtime: 66 ms

Case 1

Input

Queries =

query_name	result	position	rating
Dog	Golden Retriever	1	5
Dog	German Shepherd	2	5
Dog	Mule	200	1
Cat	Shirazi	5	2
Cat	Siamese	3	3

Code | MySQL

```
1 # Write your MySQL query statement below
2 SELECT query_name,
3 ROUND(AVG(rating/position),2) AS quality,
4 ROUND(AVG(rating < 3)*100,2) AS poor_query_percentage
5 FROM Queries
6 GROUP BY query_name;
```

Q9

Accepted 17 / 17 testcases passed
 ROSHANI DHEMBARE submitted at Sep 16, 2026 15:52

Runtime
 546 ms Beats 51.93%

Code

```

1 # Write your MySQL query statement below
2 SELECT activity_date AS day,
3       COUNT(DISTINCT user_id) AS active_users
4 FROM Activity
5 WHERE activity_date BETWEEN '2019-06-28' AND '2019-07-27'
6 GROUP BY activity_date;

```

Testcase **Test Result**
Accepted Runtime: 69 ms

Case 1

Input

user_id	session_id	activity_date	activity_type
1	1	2019-07-20	open_session
1	1	2019-07-20	scroll_down
1	1	2019-07-20	end_session
2	4	2019-07-20	open_session
2	4	2019-07-21	send_message
2	4	2019-07-21	end_session

Q10

Accepted 11 / 11 testcases passed
 ROSHANI DHEMBARE submitted at Sep 16, 2026 15:54

Runtime
 383 ms Beats 34.67%

Code

```

1 # Write your MySQL query statement below
2 SELECT class
3 FROM Courses
4 GROUP BY class
5 HAVING COUNT(student) >= 5;

```

Testcase **Test Result**
Accepted Runtime: 74 ms

Case 1

Input

student	class
A	Math
B	English
C	Math
D	Biology
E	Math
F	Computer

JOINS Problems

Q1

The screenshot shows a LeetCode submission for the problem "Replace Employee ID With The Unique Identifier". The submission is by user "ROSHANI DHEMBARE" and is marked as "Accepted". The runtime is 1198 ms, which beats 84.81% of other submissions. A bar chart shows the distribution of runtimes, with the user's submission at 1198 ms. The code is written in MySQL and uses a LEFT JOIN to replace employee IDs with unique identifiers.

Code:

```
1 # Write your MySQL query statement below
2 SELECT eu.unique_id, e.name
3 FROM Employees e
4 LEFT JOIN EmployeeUNI eu
5 ON e.id = eu.id;
```

Testcase: Accepted Runtime: 151 ms

Case 1 Input:

id	name
1	Alice
7	Bob
11	Meir
90	Winston
3	Jonathan

Q2

The screenshot shows a LeetCode submission for the problem "Product Sales Analysis I". The submission is by user "ROSHANI DHEMBARE" and is marked as "Accepted". The runtime is 1443 ms, which beats 38.62% of other submissions. A bar chart shows the distribution of runtimes, with the user's submission at 1443 ms. The code is written in MySQL and uses a JOIN to analyze product sales data.

Code:

```
1 # Write your MySQL query statement below
2 SELECT p.product_name,s.year,s.price
3 FROM Sales s
4 JOIN Product p
5 ON s.product_id=p.product_id;
```

Testcase: Accepted Runtime: 107 ms

Case 1 Input:

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Q3

The screenshot shows the LeetCode interface for the 'Employee Bonus' problem. The submission is accepted, with a runtime of 1024 ms, beating 74.29% of other submissions. A histogram shows the distribution of runtimes, with the user's submission at the peak. The code is a MySQL query that selects employee names and bonuses for employees with a bonus less than 1000 or where the bonus is null. The test result for Case 1 is shown as 'Accepted' with a runtime of 161 ms. The input data for Case 1 is a table with columns empId, name, supervisor, and salary.

Runtime: 1024 ms | Beats: 74.29%

Code:

```
1 # Write your MySQL query statement below
2 SELECT e.name,b.bonus
3 FROM Employee e
4 LEFT JOIN Bonus b
5 ON e.empId=b.empId
6 WHERE b.bonus < 1000
7 OR b.bonus IS NULL;
```

Testcase: Accepted Runtime: 161 ms

Case 1:

Input

empId	name	supervisor	salary
1	John	3	1000
2	Dan	3	2000
3	Brad	null	4000
4	Thomas	3	4000

Q4

The screenshot shows the LeetCode interface for the 'Students and Examinations' problem. The submission is accepted, with a runtime of 1264 ms, beating 28.78% of other submissions. A histogram shows the distribution of runtimes, with the user's submission at the peak. The code is a MySQL query that selects student IDs, student names, subject names, and the count of attended exams. The test result for Case 1 is shown as 'Accepted' with a runtime of 168 ms. The input data for Case 1 is a table with columns student_id and student_name.

Runtime: 1264 ms | Beats: 28.78%

Code:

```
1 # Write your MySQL query statement below
2 SELECT s.student_id,s.student_name,sub.subject_name,
3 COUNT(e.subject_name) AS attended_exams
4 FROM Students s
5 CROSS JOIN Subjects sub
6 LEFT JOIN Examinations e
7 ON s.student_id=e.student_id
8 AND sub.subject_name=e.subject_name
9 GROUP BY s.student_id,s.student_name,sub.subject_name
10 ORDER BY s.student_id,sub.subject_name;
```

Testcase: Accepted Runtime: 168 ms

Case 1:

Input

student_id	student_name
1	Alice
2	Bob
13	John
6	Alex

Q5

Managers with at Least 5 Direct Reports

Accepted 11 / 11 testcases passed

ROSHANI DHEMBARE submitted at Sep 16, 2026 16:05

School's in. Let's on. Get LeetCode Annual Premium for \$399.

Runtime: 522 ms Beats 10.58%

Code: MySQL

```
1 # Write your MySQL query statement below
2 SELECT e.name
3 FROM Employee e
4 JOIN Employee e2
5 ON e.id=e2.managerId
6 GROUP BY e.id,e.name
7 HAVING COUNT(e2.id)>=5;
```

Testcase: Accepted Runtime: 67 ms

Case 1

Input

id	name	department	managerId
101	John	A	null
102	Dan	A	101
103	James	A	101
104	Amy	A	101
105	Anne	A	101
106	Don	B	101

Q6

HackerRank | Prepare | Certify | Compete

Search

70 more points to get your first star!

Rank: 3059997 | Points: 10/80

Population Census

Problem | Submissions | Leaderboard | Discussions

RESULT	SCORE	LANGUAGE	TIME
Accepted	10.0	MySQL	a minute ago

NEED HELP?

View discussions

View top submissions

1

Q8

LeetCode submission for "Last Person to Fit in the Bus" (Problem 1413). The submission is accepted, with a runtime of 1740 ms, beating 18.27% of other solutions. The code is written in MySQL and uses a window function to calculate the cumulative weight of people boarding the bus.

Runtime: 1740 ms | Beats 18.27%

Code:

```

2 SELECT person_name
3 FROM Queue q1
4 WHERE
5 (
6     SELECT SUM(q2.weight)
7     FROM Queue q2
8     WHERE q2.turn <= q1.turn
9 ) <= 1000
10 ORDER BY turn DESC
11 LIMIT 1;

```

Testcase: Accepted Runtime: 96 ms

Case 1:

person_id	person_name	weight	turn
5	Alice	250	1
4	Bob	175	5
3	Alex	350	2
6	John Cena	400	3
1	Winston	500	6
2	Marie	200	4

Q9

LeetCode submission for "Product Price at a Given Date" (Problem 1413). The submission is accepted, with a runtime of 1665 ms, beating 5.29% of other solutions. The code is written in MySQL and uses a window function to calculate the cumulative weight of people boarding the bus.

Runtime: 1665 ms | Beats 5.29%

Code:

```

11 UNION
12
13 SELECT product_id, 10 AS price
14 FROM Products
15 WHERE product_id NOT IN
16 (
17     SELECT product_id
18     FROM Products
19     WHERE change_date <= '2019-08-16'
20 );

```

Testcase: Accepted Runtime: 69 ms

Case 1:

product_id	new_price	change_date
1	20	2019-08-14
2	50	2019-08-14
1	30	2019-08-15
1	35	2019-08-16
2	65	2019-08-17

Written Exercises

1. Explain Conceptual, Logical and Physical data models for the tables: emp, dept, meeting, address.

ANS-

A **data model** is a way of representing data, its structure, and the relationships between different data items in a database. Data models are mainly classified into **Conceptual, Logical, and Physical data models**.

Conceptual Data Model

The **conceptual data model** provides a high-level view of the database. It identifies the main entities and the relationships between them without describing technical details such as data types or SQL commands.

For the given system, the main entities are **Employee, Department, Meeting, and Address**. An employee belongs to a department, an employee can attend meetings, and an employee can have an address. A department can have many employees.

Logical Data Model

The **logical data model** describes the structure of the database in more detail. It represents entities as tables and identifies attributes, primary keys, and foreign keys. It does not depend on a particular database management system.

For example, the emp table contains employee details and dept contains department details. The deptno in the emp table can be a foreign key referring to the dept table. Similarly, tables can be created for meeting and address with appropriate keys and relationships.

Physical Data Model

The **physical data model** describes how the database is actually implemented in a particular DBMS. It specifies details such as **data types, primary keys, foreign keys, constraints, indexes, storage, and other implementation details**.

For example, empno can be defined as INT PRIMARY KEY, ename as VARCHAR(20), and sal as DOUBLE. Foreign-key constraints can be applied between emp and dept.

Difference

Conceptual model focuses on entities and relationships.

Logical model focuses on tables, attributes, keys, and relationships.

Physical model focuses on the actual implementation of the database.

2. Understanding locks: Shared locks (Read lock), Exclusive locks (Write lock), Row-level vs Table-level locks

ANS-

A **lock** is a mechanism used by a database to control simultaneous access to data by multiple transactions. Locks help maintain **data consistency and prevent conflicts** when multiple users access or modify the same data.

Shared Lock (Read Lock)

A **shared lock** is used when a transaction wants to read data. Multiple transactions can hold a shared lock on the same data at the same time. However, the data cannot be modified by another transaction using an exclusive lock while the shared lock is active.

Thus, a **shared lock is mainly associated with reading data.**

Exclusive Lock (Write Lock)

An **exclusive lock** is used when a transaction wants to modify data. When a transaction holds an exclusive lock, other transactions cannot obtain a conflicting lock on that data until the exclusive lock is released.

Exclusive locks are generally associated with operations such as **INSERT, UPDATE, and DELETE.**

Thus, an **exclusive lock is mainly associated with writing or modifying data.**

Row-Level Lock

A **row-level lock** locks only a particular row or rows of a table. Other transactions can access different rows of the same table if their operations do not conflict.

Row-level locking provides **higher concurrency** because different transactions can work on different rows simultaneously.

Table-Level Lock

A **table-level lock** locks the entire table. When a table-level lock is applied, other transactions are restricted from performing conflicting operations on the table.

Table-level locking is simpler to manage but provides **less concurrency** compared with row-level locking.

Difference Between Row-Level and Table-Level Locks

Row-level lock locks specific rows, while **table-level lock** locks the complete table. Row-level locking provides better concurrency, whereas table-level locking is simpler and has less locking overhead.

CDAC Admission Process Case Study

Design database for CDAC Admission System.

Create appropriate tables and insert sample records.

Write a Stored Procedure for Round1 allocation.

Write a Stored Procedure for Round2 allocation.

```
Command Prompt - mysql -u × Windows PowerShell × Windows PowerShell × + v

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Roshani> mysql -u KD1_Rosh_98811 -p@Roshani_10
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 3
Server version: 5.6.19 MySQL Community Server (GPL)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

KD1_Rosh_98811> CREATE DATABASE cdac_admission;
Query OK, 1 row affected (0.046 sec)

KD1_Rosh_98811> USE cdac_admission;
Database changed
KD1_Rosh_98811> CREATE TABLE candidate(
->     candidate_id INT PRIMARY KEY AUTO_INCREMENT,
->     candidate_name VARCHAR(50) NOT NULL,
->     email VARCHAR(100) UNIQUE,
->     rank_no INT UNIQUE,
->     category VARCHAR(20) DEFAULT 'OPEN',
->     age INT CHECK(age>=18)
-> );
Query OK, 0 rows affected (0.495 sec)

KD1_Rosh_98811> CREATE TABLE course(
->     course_id INT PRIMARY KEY AUTO_INCREMENT,
->     course_name VARCHAR(50) UNIQUE NOT NULL
-> );
Query OK, 0 rows affected (0.379 sec)

KD1_Rosh_98811> CREATE TABLE institute(
->     institute_id INT PRIMARY KEY AUTO_INCREMENT,
->     institute_name VARCHAR(100) UNIQUE NOT NULL,
```

```

Command Prompt - mysql -u × Windows PowerShell × Windows PowerShell × + ▾

KD1_Rosh_98811> CREATE TABLE institute(
->     institute_id INT PRIMARY KEY AUTO_INCREMENT,
->     institute_name VARCHAR(100) UNIQUE NOT NULL,
->     city VARCHAR(30) NOT NULL
-> );
Query OK, 0 rows affected (0.363 sec)

KD1_Rosh_98811> CREATE TABLE preference(
->     candidate_id INT,
->     preference_no INT,
->     course_id INT,
->     institute_id INT,
->     PRIMARY KEY(candidate_id,preference_no),
->     FOREIGN KEY(candidate_id) REFERENCES candidate(candidate_id),
->     FOREIGN KEY(course_id) REFERENCES course(course_id),
->     FOREIGN KEY(institute_id) REFERENCES institute(institute_id),
->     CHECK(preference_no>0)
-> );
Query OK, 0 rows affected (0.349 sec)

KD1_Rosh_98811> CREATE TABLE seat(
->     institute_id INT,
->     course_id INT,
->     total_seats INT DEFAULT 0,
->     available_seats INT DEFAULT 0,
->     PRIMARY KEY(institute_id,course_id),
->     FOREIGN KEY(institute_id) REFERENCES institute(institute_id),
->     FOREIGN KEY(course_id) REFERENCES course(course_id),
->     CHECK(total_seats>=0),
->     CHECK(available_seats>=0)
-> );
Query OK, 0 rows affected (0.322 sec)

KD1_Rosh_98811> CREATE TABLE allocation(
->     allocation_id INT PRIMARY KEY AUTO_INCREMENT,
->     candidate_id INT,
->     institute_id INT,
->     course_id INT,
->     round_no INT,
->     preference_no INT,

```

```

->     allocation_date DATE DEFAULT (CURRENT_DATE),
->     UNIQUE(candidate_id,round_no),
->     FOREIGN KEY(candidate_id) REFERENCES candidate(candidate_id),
->     FOREIGN KEY(institute_id) REFERENCES institute(institute_id),
->     FOREIGN KEY(course_id) REFERENCES course(course_id),
->     CHECK(round_no IN(1,2))
-> );
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '(CURRENT_DATE),
UNIQUE(candidate_id,round_no),
FOREIGN KEY(candidate_id)' at line 8
KD1_Rosh_98811> CREATE TABLE allocation(
->     allocation_id INT PRIMARY KEY AUTO_INCREMENT,
->     candidate_id INT,
->     institute_id INT,
->     course_id INT,
->     round_no INT,
->     preference_no INT,
->     allocation_date DATE DEFAULT (CURRENT_DATE),
->     UNIQUE(candidate_id,round_no),
->     FOREIGN KEY(candidate_id) REFERENCES candidate(candidate_id),
->     FOREIGN KEY(institute_id) REFERENCES institute(institute_id),
->     FOREIGN KEY(course_id) REFERENCES course(course_id),
->     CHECK(round_no IN(1,2))
-> );
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '(CURRENT_DATE),
UNIQUE(candidate_id,round_no),
FOREIGN KEY(candidate_id)' at line 8
KD1_Rosh_98811> INSERT INTO candidate(candidate_name,email,rank_no,category,age)
-> VALUES
-> ('Rahul','rahul@gmail.com',1,'OPEN',22),
-> ('Sneha','sneha@gmail.com',2,'OPEN',21),
-> ('Amit','amit@gmail.com',3,'OSC',23),
-> ('Priya','priya@gmail.com',4,'OPEN',22),
-> ('Kiran','kiran@gmail.com',5,'SC',24);
Query OK, 5 rows affected (0.075 sec)
Records: 5 Duplicates: 0 Warnings: 0

KD1_Rosh_98811> ^C
KD1_Rosh_98811> INSERT INTO course(course_name)

```

```
Command Prompt - mysql -u × Windows PowerShell × Windows PowerShell ×
KD1_Rosh_98811> ^C
KD1_Rosh_98811> INSERT INTO course(course_name)
-> VALUES
-> ('PG-DAC'),
-> ('PG-DBDA'),
-> ('PG-DITISS');
Query OK, 3 rows affected (0.042 sec)
Records: 3 Duplicates: 0 Warnings: 0

KD1_Rosh_98811> INSERT INTO institute(institute_name,city)
-> VALUES
-> ('Sunbeam Pune','Pune'),
-> ('Sunbeam Karad','Karad'),
-> ('CDAC Mumbai','Mumbai');
Query OK, 3 rows affected (0.047 sec)
Records: 3 Duplicates: 0 Warnings: 0

KD1_Rosh_98811> INSERT INTO preference VALUES
-> (1,1,1,1),
-> (1,2,2,2),
-> (1,3,3,3),
->
-> (2,1,1,1),
-> (2,2,2,2),
-> (2,3,3,3),
->
-> (3,1,1,1),
-> (3,2,2,2),
-> (3,3,3,3),
->
-> (4,1,1,1),
-> (4,2,2,2),
-> (4,3,3,3),
->
-> (5,1,1,1),
-> (5,2,2,2),
-> (5,3,3,3);
Query OK, 15 rows affected (0.050 sec)
Records: 15 Duplicates: 0 Warnings: 0

KD1_Rosh_98811> INSERT INTO seat VALUES
```